INTRODUCTION TO MACHINE LEARNING

MIDTERM PROJECT REPORT

Question 1: Linear & Logistic Regression from Scratch
Question 2: OpenML Classification Benchmark

REPORT INFORMATION	DETAILS
Course	Introduction to Machine Learning
Assessment	Middle Exam — 10 points total
Student name / ID	_____
Group / section	_____
Execution	Python 3 | NumPy | scikit-learn | Matplotlib
Prepared on	8 October 2026

EXPERIMENT SNAPSHOT

Q1: Linear Regression	Q1: Logistic Regression	Q2: Classifier comparison
R^2 0.965 / RMSE 0.646	Accuracy 88.3%	13 methods / selected 91.1% accuracy

Reproducibility note: The experiments below were actually executed and saved. Q2 uses a bundled offline copy of Fisher's Iris dataset registered at OpenML ID 61. The live OpenML endpoint could not be reached in the execution environment; this distinction is documented throughout.

Executive Summary & Submission Map

This report addresses every rubric item in the uploaded assignment: mathematical description, appropriate losses and gradients, Gradient Descent training, from-scratch implementations with evaluation for Question 1, followed by data preparation, multi-model comparison and overfitting mitigation for Question 2.

Deliverables and where to find them

Deliverable	Contents
This PDF	Theory, derivations, methodology, experimental measurements, analysis and citations
Q1_Linear_Regression.ipynb	Self-contained NumPy class code, dataset creation, training curves and test metrics
Q1_Logistic_Regression.ipynb	Stable sigmoid/BCE implementation, optimization, confusion matrix and metrics
Q2_Classification.ipynb	Dataset EDA, train-only preprocessing, 13 methods, comparative results, complexity study
src/*.py and results/*	Reusable programs and numerical output CSV/JSON files
data/* and figures/*	Bundled offline Iris data and publication-ready experimental plots

Scientific protocol

- Use deterministic random seeds and make separate training/test splits.
- Fit feature scaling only on the training data; for Q2, scale/impute inside cross-validation folds.
- Report held-out test performance, with model selection in Q2 based only on cross-validation within the training split.
- Check Q1 analytical gradients against finite-difference numerical derivatives.
- Show an explicit overfitting demonstration and a pre-pruning mitigation experiment.

Important limits

Q1 uses synthetic datasets as expressly permitted by the task. Q2 uses the Iris dataset registered at OpenML (ID 61, version 1); to ensure offline reproducibility, the observations were loaded from the local Fisher Iris copy distributed with scikit-learn. This execution did not verify a live download from OpenML. The report makes no claims about real-world decision-making reliability.

1. Linear Regression — Model & Theory

Problem: supervised regression. Given a feature vector with d numeric inputs, estimate a continuous target (e.g., house price). For a dataset of m observations, the model predicts an affine function of the features.

$$\hat{y}_i = \mathbf{x}_i^\top \mathbf{w} + b$$

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

$$\nabla_{\mathbf{w}} J = \frac{1}{m} X^\top (\hat{\mathbf{y}} - \mathbf{y})$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$

1.1 Derivation of the mean squared error gradient

Define each residual $r_i = x_i^T w + b - y_i$. The cost is $J = (1 / 2m) \sum r_i^2$. By the chain rule, the derivative of $r_i^2 / 2$ with respect to w_j is $r_i x_{ij}$. The derivative with respect to b is r_i because the intercept has coefficient one.

Therefore, $dJ/dw_j = (1/m) \sum r_i x_{ij}$, and $dJ/db = (1/m) \sum r_i$. Stacking all d derivatives gives the compact matrix expression in the formula box.

1.2 Batch Gradient Descent algorithm

- **Step 1.** Initialize w as a zero vector, b = 0, choose learning rate eta = 0.08.
- **Step 2.** In each epoch, compute all predictions and the loss on training observations.
- **Step 3.** Compute gradient_w = X^T(pred - y)/m and gradient_b = mean(pred - y).
- **Step 4.** Update w := w - eta * gradient_w; b := b - eta * gradient_b.
- **Step 5.** Repeat for 1200 epochs; never use the test set to update model parameters.

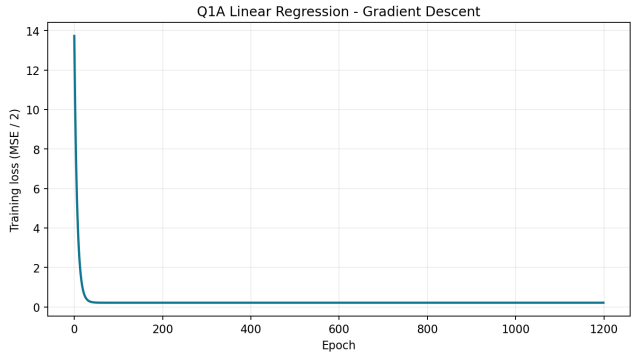
1.3 Implementation details

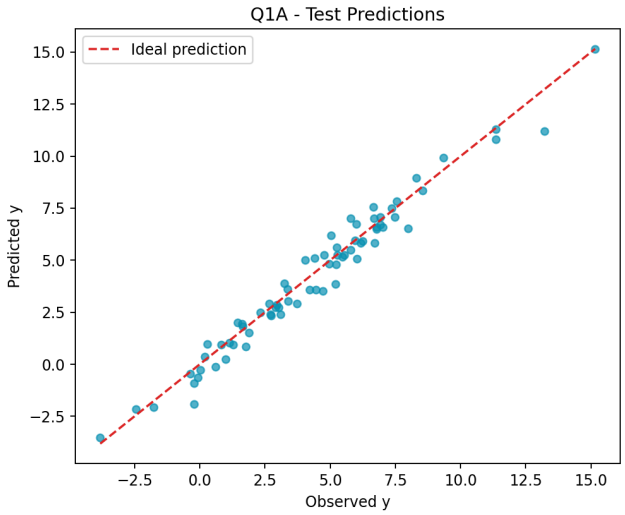
The LinearRegressionGD class in src/q1_models.py implements both the loss/gradient and update loop directly in NumPy; no scikit-learn estimator is involved in fitting. The factor of 1/2 in the loss simplifies its derivative but does not change the optimum.

2. Linear Regression — Experiment & Results

Dataset: 360 artificial observations with two independent standard-normal features and y = 4 + 3x1 - 2x2 + Gaussian noise (standard deviation 0.65). The random seed is 42. An 80/20 split produces 288 training and 72 held-out test rows. StandardScaler is fitted on the training features only.

Metric	Observed result
Learning rate / epochs	0.08 / 1200
Training loss, first -> last	13.7290 -> 0.2098
Held-out MSE	0.4176
Held-out RMSE	0.6462
Held-out MAE	0.5040
Held-out R^2	0.9655
Maximum gradient-check error	2.23e-09





Figures 1–2. The training objective decreases, and test predictions cluster around the ideal prediction diagonal.

Interpretation

The model explains approximately 96.55% of variance on the held-out test data for this deliberately linear synthetic experiment. Its RMSE of 0.646 is consistent with the injected random noise scale. The numerical gradient difference of 2.23e-09 supports the correctness of the analytical gradient implementation.

This is an illustrative result under a known data-generating process. It is not evidence that linear regression will be equally accurate for nonlinear or out-of-distribution data.

3. Logistic Regression — Model & Theory

Problem: binary classification. The score z is transformed with the sigmoid function into the estimated probability P(y=1|x). The default decision rule predicts positive class 1 when p is at least 0.5.

$$z_i = \mathbf{x}_i^\top \mathbf{w} + b \qquad p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$$

$$J(\mathbf{w}, b) = -\frac{1}{m}\sum_i [y_i \log(p_i) + (1-y_i)\log(1-p_i)]$$

$$\frac{\partial J}{\partial z_i} = p_i - y_i$$

$$\nabla_{\mathbf{w}} J = \frac{1}{m} X^\top (\mathbf{p} - \mathbf{y}), \quad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_i (p_i - y_i)$$

3.1 Why cross-entropy?

Squared error is suitable for continuous targets; for a Bernoulli outcome, the negative log-likelihood (binary cross-entropy) is the appropriate objective. It heavily penalizes confident but incorrect predicted probabilities. We implement it stably as mean(logaddexp(0, z) - y*z), avoiding log(0) and numerical overflow.

3.2 Gradient derivation: the chain rule

- For one observation, J_i = -y_i log(p_i) - (1-y_i) log(1-p_i).
- Differentiate with respect to p: dJ_i/dp = (p_i-y_i) / [p_i(1-p_i)].
- The sigmoid derivative is dp/dz = p(1-p), so these factors cancel: dJ_i/dz = p_i - y_i.
- Because dz/dw_j = x_{ij} and dz/db = 1, the model gradients are X^T(p-y)/m and mean(p-y).

3.3 Gradient Descent implementation

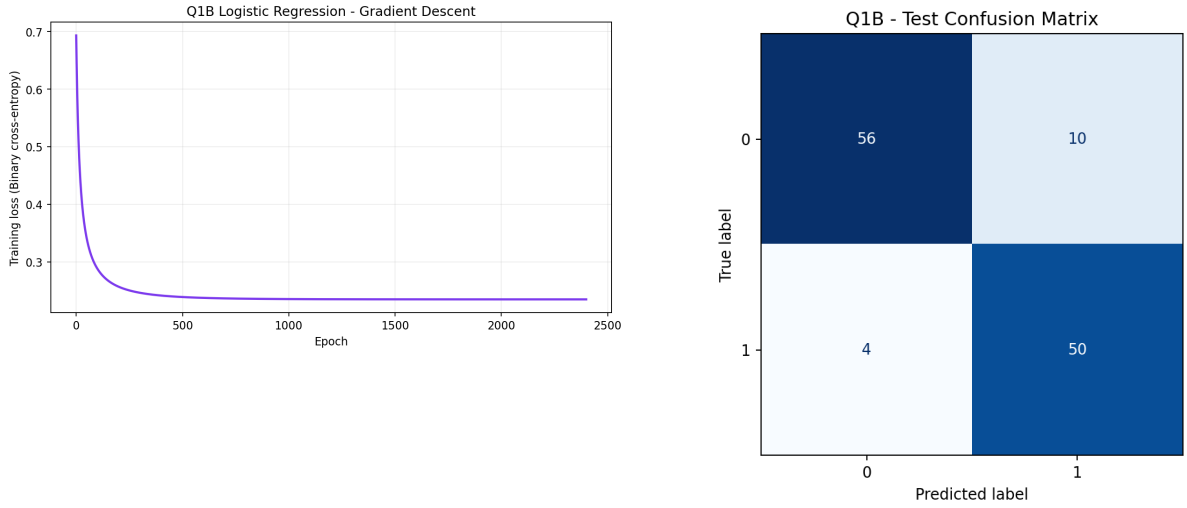
Initialize w = 0 and b = 0. At every epoch: form logits, apply a numerically stable sigmoid, calculate cross-entropy, derive residuals (p-y), and update parameters by subtracting 0.13 times the gradients. Train for 2400 epochs. The class includes a probability predictor and a configurable decision threshold.

Implementation: src/q1_models.py, class LogisticRegressionGD. The optimization algorithm is from scratch; scikit-learn is used only for generating data, preprocessing, splitting, and measuring results.

4. Logistic Regression — Experiment & Results

Dataset: 600 artificial examples, five features (four informative), binary labels, moderate imbalance (55/45), 4% label flipping, class separation 1.2. Dataset seed 27 and split seed 42. The stratified 80/20 split contains 480 training rows and 120 test rows. Only training statistics determine the scaler.

Metric	Observed result
Training loss, first -> last	0.6931 -> 0.2347
Held-out accuracy	88.33%
Precision / recall	0.8333 / 0.9259
F1 score	0.8772
ROC-AUC	0.9251
Maximum gradient-check error	8.30e-11
Confusion matrix [TN,FP;FN,TP]	[[56, 10], [4, 50]]



Figures 3–4. Cross-entropy decreases during training; the confusion matrix summarizes positive/negative test decisions.

Interpretation

Accuracy is 88.3%, while class-1 F1 = 0.877. Recall = 0.926 captures the model's ability to find positive observations, and ROC-AUC = 0.925 measures ranking across thresholds. The confusion matrix includes 4 false negatives and 10 false positives.

Accuracy alone may obscure trade-offs under class imbalance. Unlike linear regression, Logistic Regression predicts probabilities and minimizes a Bernoulli likelihood-based loss.

5. OpenML Classification — Dataset & Data Preparation

Task: predict the species of a flower from four continuous measurements. The Iris dataset is listed as **OpenML ID 61, version 1**; the listing reports 150 rows, four explanatory numeric variables, one categorical target and three classes. Source: https://www.openml.org/d/61 [1].

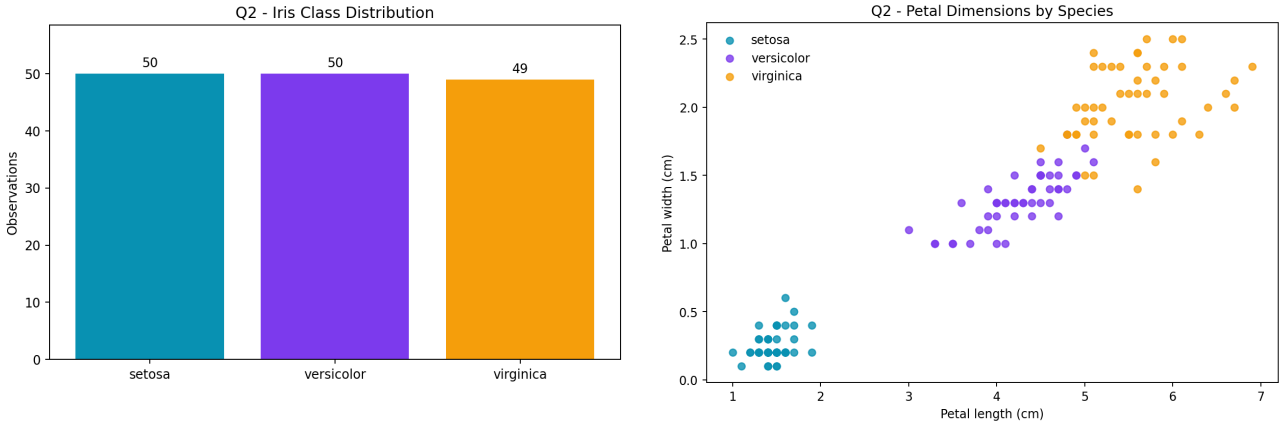
Actual execution source: the submission ships a local CSV generated from scikit-learn's Fisher Iris copy, matching the OpenML-listed problem definition. Live HTTP download was unavailable in the execution environment. Use the *--use-openml-live* option to fetch OpenML ID 61 directly when internet access is available. The local copy is not falsely presented as a live OpenML fetch.

Data property	Value
Raw rows / cleaned rows	150 / 149
Exact duplicate rows removed	1
Missing raw entries	0
Numeric features	sepallength, sepalwidth, petallength, petalwidth
Target / categories	class: setosa, versicolor, virginica
Training / test rows	104 / 45
Train/test rule	70/30 stratified, random_state = 42

5.1 Cleaning and transformations

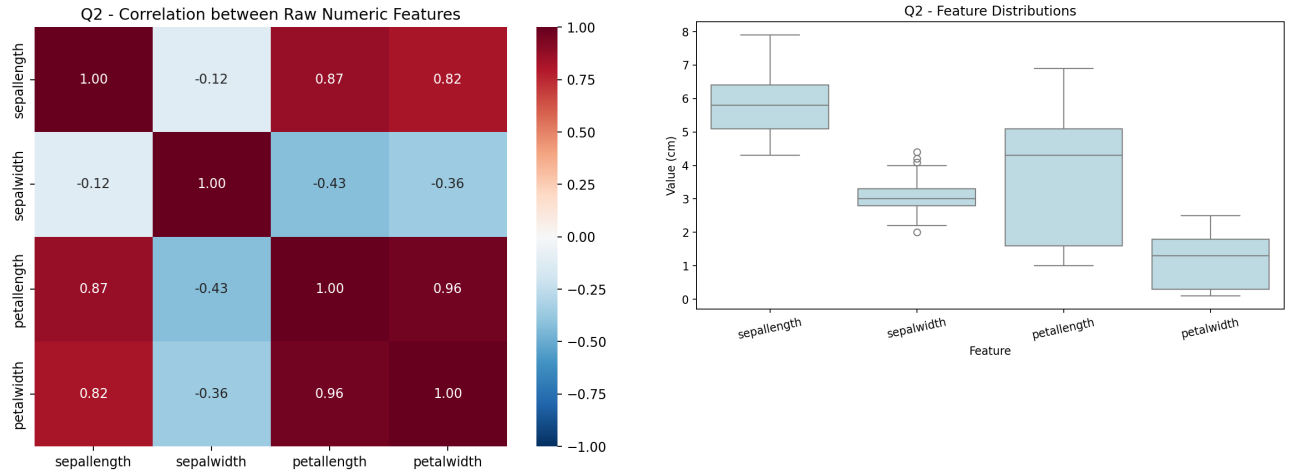
- Inspect dtypes, class distribution, missing values and exact duplicate rows; remove one fully duplicated observation.
- Convert numeric columns using pandas.to_numeric, replacing invalid or infinite entries with missing values.
- Use a numeric median imputer and StandardScaler inside a Pipeline so each cross-validation fold fits its own transformations.
- The target has three classes. The numeric features need no one-hot encoding; the labels remain categorical strings.
- Original data contain no missing values, so imputation is preventative. Do not claim missing-value imputation materially changed this dataset.

5.2 Exploratory data analysis



Figures 5–6. Flower classes are approximately balanced. Petal length and width offer visibly useful class separation.

6. Experimental Design & Data Exploration



Figures 7–8. Correlations and raw feature distributions help identify relationships and scale differences. EDA is descriptive and does not fit the predictive pipeline.

6.1 Model training and selection protocol

- After cleaning, stratify a 70% development/30% held-out split. Keep test labels inaccessible for training or CV selection.
- For each model, run 5-fold StratifiedKFold cross-validation on the training partition only, with shuffling and seed 42.
- Inside each CV fold, fit median imputation and standard scaling using only that fold's training portion; then fit the estimator.
- Rank classifiers by training-fold mean macro-F1. Ties are resolved by a reproducible alphabetic secondary order.
- Refit models on the full development split; summarize accuracy, macro precision/recall/F1 and the gap between train and held-out test accuracy.
- The final champion is identified using CV, not by reading which model had the highest held-out test accuracy.

6.2 Evaluation metric definitions

Accuracy = correct / total predictions. **Precision** = TP/(TP+FP), **Recall** = TP/(TP+FN), and **F1** is the harmonic mean of precision and recall. Macro scores average the per-class scores equally, rather than weighting them by class frequency. CV reports fold mean and standard deviation.

6.3 Methods compared

Dummy majority-class predictor; Logistic Regression; k-Nearest Neighbors; SVM (RBF and linear); Decision Tree; Random Forest; Extra Trees; Gradient Boosting; AdaBoost; Gaussian Naive Bayes; Linear Discriminant Analysis; Quadratic Discriminant Analysis (13 methods total). Hyperparameters are deterministic fixed baselines for a broad comparison, not exhaustive model tuning.

7. Classification Benchmark — Measured Results

The table below reports actual training-only 5-fold cross-validation macro-F1 and held-out test metrics. Ranking is computed using training CV macro-F1, not test metrics. Scores are rounded to three decimals for readability.

Method	CV macro-F1	Test accuracy	Test macro-F1
Gaussian Naive Bayes	0.981 ± 0.023	0.911	0.911
Linear SVM	0.981 ± 0.023	0.867	0.868
Logistic Regression	0.981 ± 0.023	0.911	0.911
Extra Trees	0.971 ± 0.038	0.911	0.911
LDA	0.971 ± 0.038	0.978	0.978
SVM (RBF)	0.971 ± 0.038	0.911	0.911
QDA	0.971 ± 0.023	0.978	0.978
KNN (k=5)	0.970 ± 0.024	0.911	0.910
Gradient Boosting	0.961 ± 0.037	0.933	0.933
AdaBoost	0.952 ± 0.052	0.933	0.933
Decision Tree	0.952 ± 0.052	0.911	0.911
Random Forest	0.952 ± 0.052	0.911	0.911
Dummy (majority)	0.168 ± 0.002	0.333	0.167

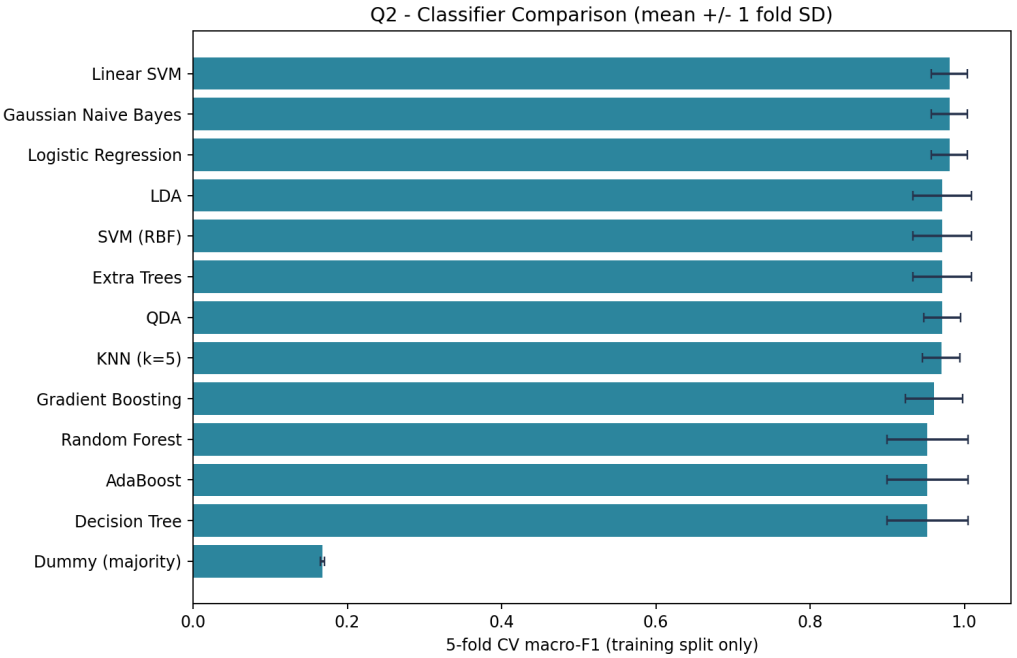


Figure 9. Mean training-fold macro-F1 scores with one standard-deviation error bars.

8. Final Selection & Hold-Out Evaluation

By the prespecified CV criterion, the selected model is **Gaussian Naive Bayes**. Its held-out accuracy is **0.9111** and macro-F1 is **0.9107**. The held-out sample contains 45 observations.

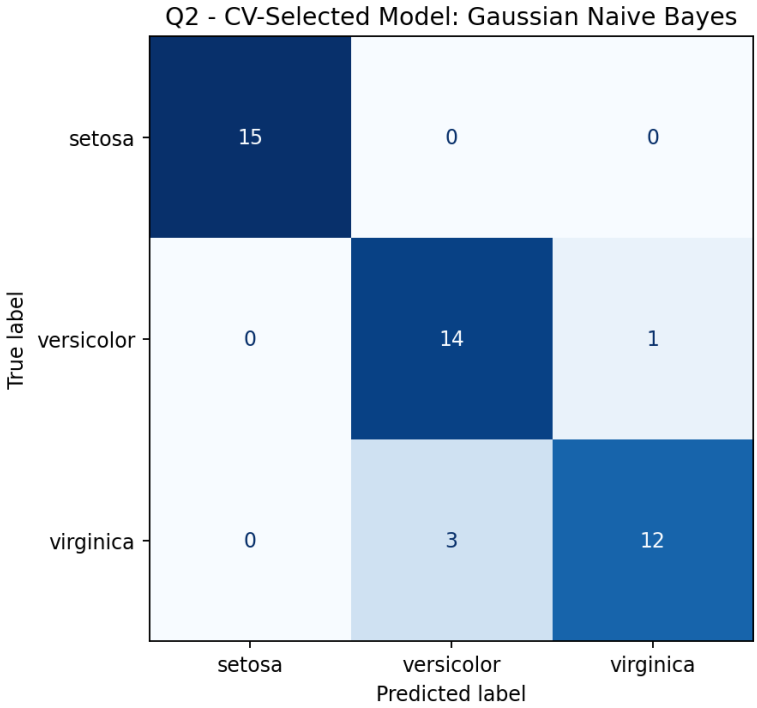


Figure 10. Confusion matrix for the CV-selected classifier; rows are actual classes, columns are predictions.

Why the highest test score is not automatically the selected model

A few candidates (e.g., LDA and QDA in this particular split) show higher measured test scores than the selected model. Choosing a replacement based on those same test outcomes would turn the test set into a tuning signal. This report therefore respects the prespecified training-only cross-validation criterion and uses the test split only as an independent performance estimate.

Statistical caution

The sample size is modest; the test set contains only 45 records. A difference of one correctly classified flower changes test accuracy by about 2.22 percentage points. Single-split rankings can therefore be unstable. A more comprehensive study would repeat nested cross-validation and quantify uncertainty, using a larger independent test collection where possible.

The experiment demonstrates standard classification workflows, not deployable flower-identification software.

9. Overfitting — Diagnosis & Mitigation

Overfitting happens when a model fits the development set so closely that its performance on new data is weaker. A highly flexible Decision Tree can isolate individual training examples and achieve near-perfect in-sample performance, even when the boundary is unnecessarily complex.

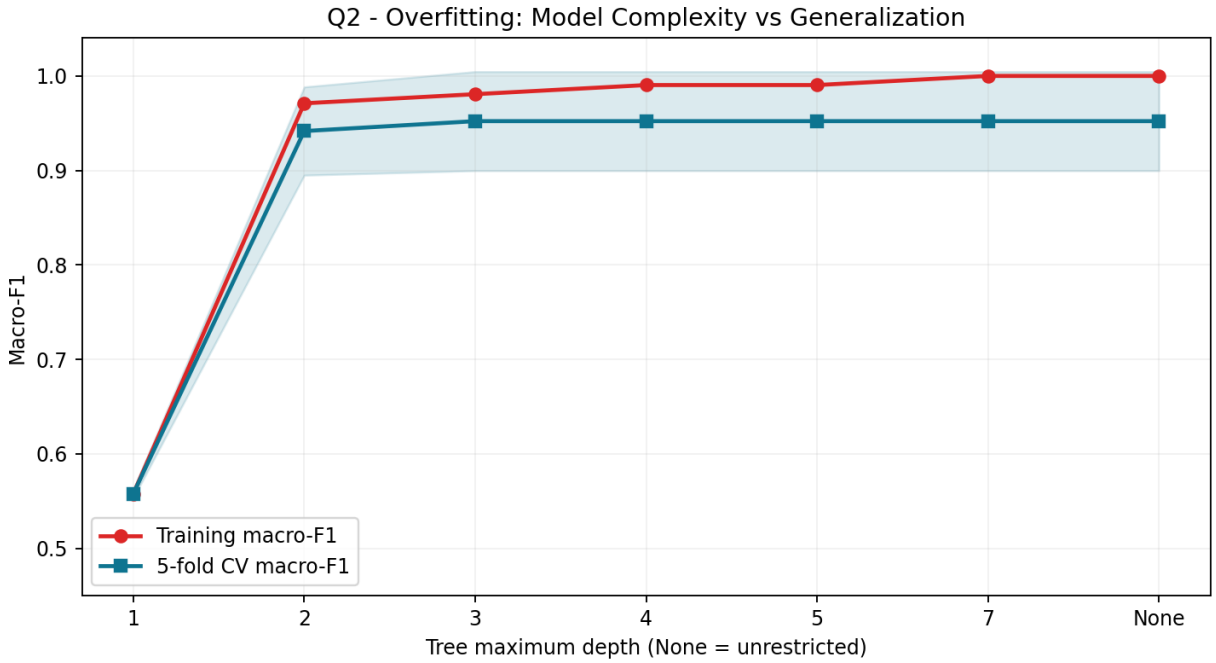


Figure 11. Train-vs-cross-validation macro-F1 across maximum tree depths; the shaded area marks CV fold variation.

9.1 Observed generalization gap

The unrestricted tree achieves train macro-F1 = **1.0000** but hold-out macro-F1 = **0.9111**, consistent with an overfitting gap. Cross-validation scores plateau while training fit increases at greater depth.

9.2 Demonstration: early stopping via tree pre-pruning

Select max_depth from the candidates [1, 2, 3, 4, 5, 7, unrestricted] using mean 5-fold macro-F1 on training data only, with simpler trees favored in a tie. The chosen depth is **3**. The resulting train macro-F1 is **0.9807** and hold-out macro-F1 is **0.9327**.

Model	Train macro-F1	Test macro-F1
Unrestricted Decision Tree	1.0000	0.9111
Pre-pruned tree (depth 3)	0.9807	0.9327

The reported improvement is empirical for this one split; it is not a guarantee that tree pruning always improves held-out accuracy.

10. Overfitting Remedies by Algorithm & Conclusions

10.1 Model-specific prevention strategies

Method family	Practical overfitting controls
Logistic Regression / linear SVM	Increase regularization (reduce C), select relevant features, tune on CV
kNN	Increase k when k=1 is noisy, scale inputs, tune k inside CV
RBF SVM	Tune C and gamma using training folds; avoid unnecessarily complex boundaries
Decision Tree	Limit max_depth, min_samples_leaf, min_samples_split, use cost-complexity pruning
Random Forest / Extra Trees	Control maximum depth and leaf size; consider max_features and cross-validation
Boosting	Lower learning rate, fewer stages, shallower weak learners, early stopping
Naive Bayes / LDA / QDA	Inspect assumptions, regularize covariance estimates where appropriate

10.2 Synthesis of the two exam questions

Question 1 demonstrates analytic gradients, correct batch Gradient Descent implementation and finite-difference verification. Regression predicts numeric targets with squared-error loss; binary Logistic Regression predicts probabilities using cross-entropy. Their performance was measured on untouched held-out sets.

Question 2 demonstrates reproducible data analysis and leakage-safe classification. Thirteen algorithms were evaluated under the same split and CV protocol. The Decision Tree complexity experiment shows why training performance alone is not a sufficient quality measure. Pre-pruning a tree by a training-only CV selection rule improved the observed held-out macro-F1 in this run.

10.3 Limitations and ethical considerations

- Both Q1 datasets are synthetic, so their success may not transfer to noisy real-world tasks.
- The Iris dataset is small and nearly separable; a 45-row held-out split yields high variability.
- The offline Iris copy was not fetched from OpenML live at execution; the corresponding OpenML ID is cited and the optional live-fetch path is supplied.
- Tree depth was tuned on training CV, but comparing many test scores in a report may invite post-hoc selection; use an independent final evaluation for deployment decisions.
- Data preprocessing and evaluated models should be tailored to data context; performance differences on Iris are not universal model rankings.

11. Reproducibility, Files & References

11.1 Suggested procedure (from extracted project directory)

1. Create a Python virtual environment and install required packages with **pip install -r requirements.txt**.

2. Run **python -m src.q1_models** to regenerate both from-scratch optimizations, four plots and results/q1_metrics.json.

3. Run **python -m src.q2_classification** to regenerate EDA, classifier benchmark, tree depth study, results tables, and JSON summary.

4. Open the three Jupyter notebooks in the notebooks/ folder, or use **jupyter lab** from the project root. Notebook outputs have been precomputed and saved.

5. For a direct OpenML download where the network is available, execute **python -m src.q2_classification --use-openml-live**. This flag intentionally errors if the download fails instead of silently claiming online provenance.

11.2 Saved data and evidence

Path	Contents
data/iris_openml_61_offline_mirror.csv	150-row offline Fisher Iris mirror; one exact duplicate removed during cleaning
results/q1_metrics.json	Actual held-out metrics, learning curves summary and numerical gradient checks
results/q2_model_comparison.csv	13-model CV and held-out comparison
results/q2_tree_depth_study.csv	Depth-versus-training/CV macro-F1 experiment
results/q2_summary.json	Cleaning, champion, confusion matrix and mitigation metadata
figures/*.png	Saved visual evidence corresponding to plots in this PDF
src/*.py	Runnable training programs without hidden external dependencies

11.3 References

[1] OpenML, *iris* (dataset ID 61, version 1), https://www.openml.org/d/61. Accessed October 2026.

[2] OpenML Documentation, *Datasets / Download a dataset*, https://docs.openml.org/examples/20_basic/simple_datasets_tutorial/.

[3] R. A. Fisher (1936), *The Use of Multiple Measurements in Taxonomic Problems*, Annals of Eugenics 7(2), 179-188.

[4] scikit-learn documentation, *Dataset loading / load_iris; Pipeline and cross-validation*, https://scikit-learn.org/stable/.

Implementation details, seeds, metric definitions, source code, and saved numerical results are supplied in the companion ZIP archive.